ABAP INTERVIEW PREPARATION

Senior ABAP Cloud mock interview

Six connected cases, code diagnosis, model reasoning and a 75-minute self-check

329 questions across 24 study segments. Reviewed 11 October 2026.

Use the short answers for oral rehearsal. The examples and pitfalls prepare you for follow-up questions. Choose the target SAP edition and release before applying an implementation detail.

Read online in GitHub through README.md, or download the repository and open START_HERE.html for offline recall cards. The PDF is a portable reading and printing edition.

This 75-minute interview follows one purchase-approval extension from architecture to a production incident. It tests decisions and failure reasoning rather than memorized definitions. The scenarios are original practice cases. Establish the target edition, ABAP release, language version and consumer before promising support.

All five code blocks are intentionally incomplete teaching fragments, some deliberately defective. Names are hypothetical. They are not compilable repository objects or verified executable APIs. Validate real syntax, types, API release state and runtime behavior in the target SAP system before implementation.

Interview schedule

Segment	Minutes	Cumulative
Read context and clarify assumptions	3	3
Exercise 1: architecture and Clean Core	11	14
Exercise 2: partial updates and rejection	12	26
Exercise 3: security across layers	11	37
Exercise 4: custom query correctness	10	47
Exercise 5: posting and transaction ownership	11	58
Exercise 6: concurrency and recovery	12	70
Review evidence gaps and score	5	75

Shared context: employees create purchase requests with items. Approvers are restricted by company code and approval limit. A supplier portal reads approved requests. Posting creates an SAP purchasing document. Requests support draft editing. One integration dependency may be unavailable, and users may retry operations after timeouts.

Scoring rubric

Score four dimensions from 0 to 3 for each exercise: boundary, mechanism, trade-off and evidence. The maximum is 12 per exercise and 72 overall. These are self-assessment criteria, not an employer's hiring scale.

Score	Boundary	Mechanism	Trade-off	Evidence
0	Misses transaction or trust boundary	Incorrect or absent mechanism	No alternatives considered	Claims success without verification
1	Names a boundary without tracing it	Names a relevant feature	Names a cost without deciding	Proposes only a happy-path test
2	Explains ownership and failure path	Applies the right supported contract	Defends a choice against an alternative	Defines observable negative tests
3	Handles cross-layer and retry effects	Qualifies release limits and exceptions	Links decision to business consequences	Specifies fault injection, assertions and operational evidence

Do not average away a dangerous misconception. Independent commits inside RAP behavior, authorization bypass, silently ignored filters or assumed remote atomicity require correction even if the total is high. For readiness, aim for at least 2 in every dimension on every case, then repeat weak cases aloud without reading the model answer.

Exercise 1 Design the extension and defend its Clean Core position

The architect proposes one on-stack RAP BO containing request, items, supplier portal sessions and the eventual standard purchase order. A developer says it is automatically Clean Core Level A because RAP is used. The only familiar posting function is not released for ABAP Cloud consumption.

Explain the BO boundary, deployment placement and dependency decision. Identify which missing facts could change your design, and give the sponsor a practical path forward.

Your answer before reading the solution

Model answer

Keep request and items together where their lifecycle, locking and invariants belong together. Treat the standard purchase order as another business object accessed through an appropriate API. Portal identity and external session concerns are separate. Prefer managed behavior for suitable new custom persistence, while choosing on-stack versus side-by-side placement according to required consistency, API availability, latency and independent lifecycle.

Clean Core classification follows actual dependencies and extension mechanisms, not the RAP label. Inspect the posting API's release/classification and the target edition. Seek a released operation or supported extension first. A custom wrapper does not erase an underlying dependency's risk. If a constrained classic path is considered in a suitable edition, record its owner, upgrade evidence, expiry and replacement plan; do not present it as generally available in ABAP Cloud.

Probes

What requirement genuinely needs the same local transaction? What changes if only a remote posting API exists? How would you justify a temporary dependency exception without calling it Level A?

Wrong answers to challenge

"RAP makes every dependency cloud compliant." "A side-by-side application has no core dependencies." "A wrapper converts any internal object into a released SAP API."

Evidence and scoring focus

Draw trust and transaction boundaries. Produce a dependency inventory with target API states, one architecture decision and an upgrade test. Score the distinction between placement, language restrictions and Clean Core classification.

Study links: CLOUD-01, CLOUD-05 and CLOUD-07 in cloud architecture; GOV-01, GOV-03 and GOV-07 in Clean Core governance.

Exercise 2 Diagnose a partial update and a validation that does not reject

A caller clears an approver comment, but its previous text survives. Another request receives an error message yet still saves an invalid amount. Diagnose both fragments and explain how a bulk request containing valid and invalid items should be tested.

```
" Defective teaching fragment: context and types omitted.
" The caller intends to clear Comment for an existing request.
ls_update-Comment = ''.
ls_update-%control-Comment = if_abap_behv=>mk-off.
" ls_update is subsequently passed to EML UPDATE FROM.

" Defective validation fragment: generated signature omitted.
" ls_reported contains an error message and the instance identity.
APPEND ls_reported TO reported-Request.
" The rejected instance is never appended to failed-Request.
```

Your answer before reading the solution

Model answer

An initial field value and an omitted update are different. The first fragment marks Comment as not supplied. Use the intended field list or correct control flag; do not mark all fields indiscriminately, because omitted values could then overwrite unrelated data. Inspect adapters and defaulting determinations as additional places where an explicit clear can be lost.

In the validation, report the failing instance through the permitted FAILED contract and associate its message with the correct transactional identity, field and path. REPORTED alone is not a universal rejection mechanism. Determinations have a different signature and do not offer FAILED. For bulk processing, handle every input and preserve correlation; the surrounding transaction owner and protocol boundary determine the intended atomicity policy.

Probes

How do omitted Amount and Amount equal to zero differ? Which identity components matter for drafts? Does one FAILED entry always roll back every operation in an entire OData batch?

Wrong answers to challenge

"IS INITIAL means the field was omitted." "An error-severity message always stops saving." "Only the first failed item matters."

Evidence and scoring focus

Test omitted, empty and nonempty Comment; confirm untouched fields survive. Submit several items with two different validation failures and assert their identities, messages and final persistence. Distinguish interaction responses from save outcome.

Study links: SRAP-05, SRAP-06, SRAP-07 and SRAP-20 in RAP transaction scenarios.

Exercise 3 Repair read and operation authorization together

The base request view has DCL. A new service projection exposes requests from other company codes. The UI hides Approve for ordinary users, but a direct action request still succeeds. A third user can update a permitted request so that its CompanyCode becomes an unauthorized one.

```
// Incomplete CDS teaching fragment, not a complete model.
@AccessControl.authorizationCheck: #CHECK
define view entity ZC_Request
  as projection on ZI_Request
{
  key RequestId,
      CompanyCode,
      Amount
}
// DCL is known only for ZI_Request; none is shown for ZC_Request.
```

Your answer before reading the solution

Model answer

Check access control on the actual consumed entity. The annotation requests access-control checking; it does not create a rule or automatically inherit the base rule. Define the projection's protection using a suitable supported DCL design or explicit inheritance with correct mappings. Test service reads, navigation and value helps, not merely the base entity.

RAP operation authorization must protect Approve independently of button visibility. Feature control expresses business availability, such as preventing approval of cancelled requests. A company-code change also needs a decision about the proposed target context; checking only the original instance can leave a privilege gap. Use the appropriate incoming-change and authorization mechanisms while preserving supported phase and partial-update semantics.

Probes

Can DCL alone authorize Approve? What checks can IN LOCAL MODE bypass, and where is that mode legitimate? How would you prevent information leakage through value helps or counts?

Wrong answers to challenge

"#CHECK creates authorization automatically." "Hidden buttons protect the API." "Any authorized update may move data to any company code."

Evidence and scoring focus

Use permitted and denied users with known foreign keys. Test reads, direct actions and A-to-B company changes. Compare UI behavior with backend rejection and ensure negative tests assert absence of data, not merely an error banner.

Study links: MODEL-01, MODEL-02, MODEL-11 and MODEL-13 in CDS and security; SRAP-12 in RAP scenarios.

Exercise 4 Make a remote custom query correct before optimizing it

The supplier portal reads a custom entity. A complex filter produces an exception; the developer catches it and returns everything. The count is the number of rows on the current page, and no stable sort is applied.

```
DELIBERATELY DEFECTIVE PSEUDOCODE; NOT ABAP
try: filter = convert_request_to_simple_ranges()
catch unsupported_filter: filter = no_filter
rows = fetch_remote_rows(filter)
page = first_requested_slice(rows)
count = length(page)
return page, count
```

Your answer before reading the solution

Model answer

Never broaden a query silently when filter conversion fails. Preserve the complete supported filter semantics through an appropriate representation, or reject an unsupported request explicitly. Apply authorization to both returned data and count. Count represents the filtered authorized result before paging, not current page length; calculate it only when requested.

Use a deterministic order with a suitable tie-breaker and push supported filtering, sorting and paging to the remote provider. Stable ordering alone does not guarantee a frozen result while data changes between pages; explain the source's consistency and continuation-token contract. Avoid fetching all records merely to discard most locally, and do not promise an exact cross-page snapshot that the remote API cannot provide.

Probes

What happens with an OR expression that simple ranges cannot represent? How would you process count-only requests? What changes when records are inserted between pages?

Wrong answers to challenge

"Returning more rows is a harmless fallback." "Count equals page size." "ORDER BY any nonunique field prevents all paging anomalies."

Evidence and scoring focus

Use complex filters, no matches, count-only, second-page and concurrent-insert cases. Assert authorization and total count. Measure remote call count and transferred rows under representative volume.

Study links: MODEL-03, MODEL-04 and MODEL-05 in CDS and security; OPS-07 and OPS-08 in production readiness.

Exercise 5 Integrate posting without losing transaction ownership

The local prototype reaches a nested COMMIT WORK and violates the RAP runtime contract. A proposed RFC workaround commits a purchasing document remotely, but the local request later fails validation. The developer proposes moving posting to SAVE.

```
" Defective local prototype: forbidden nested commit.
" A posting API was called from a RAP behavior implementation.
" Deeper in that call chain, legacy code executes:
COMMIT WORK.
" The RAP request has not completed its own save decision.
```

Your answer before reading the solution

Model answer

The forbidden local commit is rejected by the RAP runtime contract; do not assume it succeeds and validation then continues normally. A separately committed remote effect is a different failure mode: local rollback cannot undo it. A wrapper or different caller phase does not remove the internal commit. Identify the actual transaction owner and audit the call chain for explicit and implicit boundaries. Prefer an eligible API whose transaction contract fits the scenario. If local atomic integration cannot be supported, model a durable posting request and explicit asynchronous processing with reconciliation rather than claiming one LUW.

Do not generalize ordinary business-error handling into the late save phase. Perform predictable checks earlier. Equally, avoid an absolute claim that supported late failure handling never exists: SAP documents a failure-capable saver for specific BAPI integrations, including late numbering when the create call returns the final key. Verify that precise release, saver, API and authorization contract; it does not authorize arbitrary committing legacy code.

Probes

What if the final document number appears only after posting? What if the remote call times out after succeeding? Which state allows operations staff to reconcile an uncertain result safely?

Wrong answers to challenge

"Unmanaged save permits any COMMIT." "An RFC wrapper restores atomicity." "Retry every timeout as a new create."

Evidence and scoring focus

Trace the first durable effect. Inject failures before and after posting, confirm numbering correlation, and replay an uncertain result. Explain how duplicate documents are prevented and how support establishes the actual outcome.

Study links: SRAP-03, SRAP-04, SRAP-08 and SRAP-15 in RAP scenarios; OPS-11 in production readiness.

Exercise 6 Resolve a stale draft and duplicate approval event

A user leaves an edit draft open. Its exclusive lock expires. Another process changes an active item. The user resumes and submits; two nearly simultaneous Submit requests arrive. The approval event is delivered twice, while the UI still shows the previous total.

Separate the failures. Propose the smallest coherent corrections and a release gate that would detect their recurrence.

Your answer before reading the solution

Model answer

First verify the total ETag and lock model across the composition. Child-only active changes must participate in the required version protection so a stale draft is detected when resumed. Ordinary entity ETags and draft total ETags address related but distinct concurrency checks.

Make Submit a protected state transition with a defined outcome for already submitted requests. Coordinate durable follow-up intent with the transition and make external consumers idempotent using a stable business/event identity. A timeout can mean an unknown outcome, not a failed transition. Finally, diagnose the displayed total separately: verify backend transactional values, then inspect side effects, action results and actual refresh requests. A UI refresh fix must not introduce another commit.

Probes

How do you test genuine concurrency with two sessions? What if a cancellation occurs before a delayed approval event arrives? Which evidence survives a business rollback without leaking sensitive data?

Wrong answers to challenge

"A lock protects a draft forever." "Events are always delivered exactly once." "Refreshing the whole page proves the backend calculation is correct."

Evidence and scoring focus

Run child-only change after lock expiry, concurrent Submit, response loss, duplicate and stale event delivery, and UI refresh checks. Assert one effective transition and intended external effect. The release gate includes backend negative tests, actual consumer behavior, monitoring, deployment prerequisites and recovery ownership; passing ATC alone is insufficient.

Study links: SRAP-10, SRAP-18, SRAP-21 and SRAP-23 in RAP scenarios; OPS-05, OPS-06 and OPS-14 in production readiness.

Five-minute debrief

Record each exercise's four scores and the weakest claim you made. Replace vague phrases such as "the framework handles it" with the named owner, relevant contract and observable assertion. Select three unanswered probes for targeted study. Repeat the interview after two days with changed assumptions: remote instead of local posting, an older target release, or a direct API consumer instead of Fiori.